

ILB Incubator Management Platform

Final Assignment Report

SDL Project Group 20

May 2026

Abstract

This report summarises the final delivery of the ILB Incubator Management Platform: a Docker Compose deployable microservices system for managing incubated companies, contracts, finance, spaces, bookings, inventory, support tickets, and documents. The final branch consolidates strict service boundaries, role-aware frontend workflows, RabbitMQ event integration, operational runbooks, and a GitLab CI/CD path for variable-driven production deployment.

Contents

1	Project scope	2
2	Architecture overview	2
3	Implemented capabilities	2
3.1	Backend services	2
3.2	Frontend workflows	3
3.3	Event-driven integration	3
4	Runtime and deployment	3
4.1	Docker Compose runtime	3
4.2	GitLab CI/CD production deployment	3
5	Quality evidence	4
6	Known limitations and operational notes	4
7	Conclusion	5

1 Project scope

The platform supports incubator staff, client-company users, and public booking requesters. Staff users operate the internal dashboard, companies, contracts, finance, spaces, bookings, inventory, tickets, users, and documents. Client users access company-scoped contracts, payments, bookings, tickets, and company profile information. Public users can submit booking requests without an authenticated session.

The final objective was to deliver a production-shaped system that can be run locally with Docker Compose, explained during the defense, and promoted through GitLab CI/CD without committing environment-specific secrets.

2 Architecture overview

The repository is a monorepo, but the runtime model is deliberately microservice-oriented. Each bounded context is a separate Django application and owns its own PostgreSQL database. Services communicate through REST for request/response flows and RabbitMQ events for asynchronous consistency.

Compose service	Bounded context	Gateway prefix	Port
auth-service	Identity, sessions, JWT issuance and validation	/api/auth/	8001
company-service	Incubated company profiles, CAE, maturity and archive state	/api/companies/	8002
contract-service	Company contracts and lifecycle	/api/contracts/	8003
finance-service	Payments, billing, overdue processing, reports	/api/finance/	8004
space-service	Physical spaces, capacity and occupancy projections	/api/spaces/	8005
booking-service	Staff, client and public booking workflows	/api/bookings/	8006
inventory-service	Equipment catalogue, assignment and release	/api/inventory/	8007
ticket-service	Support queues and ticket threads	/api/tickets/	8008
dashboard-service	Cross-service operational aggregates	/api/dashboard/	8009
document-service	Document metadata and MinIO-backed object storage	/api/documents/	8010

The Nginx gateway is the external HTTP boundary. It serves the Next.js frontend and routes API traffic by path prefix. Authentication uses gateway validation against the auth service and forwards trusted identity headers to downstream services. Internal Django service ports remain private to the Compose network.

3 Implemented capabilities

3.1 Backend services

The final implementation covers the main demo path across all service domains:

- Auth: login, JWT handling, user management, gateway verification and RBAC.
- Company: company CRUD, maturity/status data, employees, statistics and events.

- Contract: lifecycle endpoints, monthly fee snapshots, expiration handling and events.
- Finance: payments, billing/overdue commands, event consumers and reports.
- Space: type and space CRUD, occupancy, contract and booking projections.
- Booking: staff/client/public booking flows and lifecycle events.
- Inventory: equipment/type CRUD, assignment/release and booking event handlers.
- Ticket: scoped tickets, message threads and staff metrics.
- Dashboard: service health, ticket KPIs and aggregate snapshots.
- Document: metadata plus MinIO-backed upload, download, list and delete flows.

3.2 Frontend workflows

The frontend is a Next.js 14 App Router application with TypeScript and Ant Design. It separates staff routes from client portal routes and keeps the public booking request outside the authenticated shell. The delivered pages support the defense walkthrough: staff dashboard, company/contract/finance/spaces/bookings/ inventory/tickets/users areas, client portal pages, and public booking request.

3.3 Event-driven integration

Cross-service consistency is implemented with the `incubator.events` RabbitMQ topic exchange and a shared envelope from `libs/py-common`. Events include `contract.activated`, `contract.terminated`, `booking.approved`, `booking.cancelled`, `booking.completed`, and `payment.recorded`. Consumers persist or check `event_id` values so duplicate delivery can be retried safely. Recurring work is implemented with scheduler sidecar containers that run Django management commands from cron files; the design intentionally avoids Celery.

4 Runtime and deployment

4.1 Docker Compose runtime

The production-shaped runtime is defined in `infra/docker-compose.yml`. It includes PostgreSQL, Redis, RabbitMQ, MinIO, one Django container per bounded context, scheduler/consumer sidecars where needed, the Next.js frontend, and the Nginx gateway. Environment variables are sourced from a root `.env` file; `.env.example` documents the required non-secret defaults and values to review before deployment.

4.2 GitLab CI/CD production deployment

The final branch includes a variable-driven production deployment path in `.gitlab-ci.yml` and `infra/scripts/ci/deploy-remote.sh`. The manual `deploy:production` job runs on `main`, uses protected GitLab variables for SSH access and the production environment file, archives the current commit to the server, validates the Compose configuration, and runs Docker Compose remotely.

Changed-service selection is handled by `infra/scripts/ci/detect-changed-services.sh`. Service-local changes rebuild only the affected Compose services and sidecars. Shared library, Compose, deploy script, or environment-template changes trigger a full application rollout to avoid stale dependent images.

Listing 1: Production deploy command shape

```
docker compose --env-file .env --project-directory . \
-f infra/docker-compose.yml config --quiet

docker compose --env-file .env --project-directory . \
-f infra/docker-compose.yml up -d --build --remove-orphans \
<changed-services>
```

Required GitLab variables are `PRODUCTION_DEPLOY_HOST`, `PRODUCTION_DEPLOY_USER`, `PRODUCTION_DEPLOY_PA`, `PRODUCTION_DEPLOY_SSH_PRIVATE_KEY`, and either `PRODUCTION_ENV_FILE_BASE64` or `PRODUCTION_ENV_FILE`. No production hostnames, keys, or secrets are committed to the repository.

5 Quality evidence

The repository contains defense and release evidence under `docs/defense/`. The latest recorded release criteria require host-only quality gates, Docker-backed Compose evidence on a host with Docker socket access, frontend typecheck/unit/lint/build, browser smoke, and updated operator documentation.

For this final delivery task, the following checks were rerun in the worker branch:

- `bash -n infra/scripts/ci/*.sh`
- `docker-compose -f infra/docker-compose.yml -project-directory . config -quiet`
- GitLab YAML parse for `.gitlab-ci.yml` and child CI files.
- Changed-service selector smoke tests in a temporary Git repository.
- `npx prettier -check .gitlab-ci.yml docs/deploy.md`
- `npm -prefix frontend run typecheck`
- `python3 -m pytest infra/tests -q` with 15 passing tests.
- `latexmk -pdf -interaction=nonstopmode -halt-on-error docs/final-report/final-report.tex`

6 Known limitations and operational notes

- Live Compose and gateway Playwright evidence require a host that can access the Docker socket. If unavailable, the blocker must be recorded as an environment limitation rather than product evidence.
- The production deploy job was validated structurally and by script checks, but a live SSH deployment requires protected GitLab variables and a prepared Docker host.
- Dependency audit warnings are present in the existing frontend lockfile and should be handled in a separate dependency/security maintenance lane.
- Production secrets must stay in GitLab protected variables or a secrets manager; the repository should never contain real `.env` values.

7 Conclusion

The final assignment delivery provides a coherent, demonstrable incubator management platform with strict service ownership, event-driven integration, role-aware frontend workflows, Compose-based runtime operations, and a variable-driven GitLab production deployment route. The included documentation, defense scripts, CI/CD configuration, and compiled report support both technical review and operational handoff.